

Manual for Artificial Intelligence and Applications

Lab BCA VI Sem – Final AI Lab Programs

– Arshad Pasha

All Code , For Studying 🤪 :

<https://github.com/pashaarshad/AI-LAB-6th-Sem-2026>

1. Write a program to implement breadth first search using python.

```
2. g={
3.     'A':['B','C'],
4.     'B':['D','E'],
5.     'C':['F'],
6.     'D':[],
7.     'E':['F'],
8.     'F':[]
9. }
10.
11. v = []
12. q = []
13.
14. def bfs(v,g,n):
15.     v.append(n)
16.     q.append(n)
17.
18.     while q:
19.         s=q.pop(0)
20.         print(s,end=" ")
21.
22.         for nib in g[s]:
23.             if nib not in v:
24.                 v.append(nib)
25.                 q.append(nib)
26. bfs(v,g,'A')
27.
```

Output :

```
PS C:\Users\Admin\Desktop\BCA-6sem\AI> & "C:\Program Files\Python313\python.exe" c:/Users/Admin/Desktop/BCA-6sem/AI/p1.py
ABCDEF
```

2. Write a Program to Implement Depth First Search using Python.

```
28. g={
29.     'A':['B','C'],
30.     'B':['D','E'],
31.     'C':['F'],
32.     'D':[],
33.     'E':['F'],
34.     'F':[]
35. }
36. v = set()
37.
38. def dfs(v,g,n):
39.     if n not in v:
40.         print(n,end=" ")
41.         v.add(n)
42.
43.         for n in g[n]:
44.             dfs(v,g,n)
45.
46. dfs(v,g,'A')
```

Output :

```
PS C:\Users\Admin\Desktop\BCA-6sem\AI> & "C:\Program Files\Python313\python.exe" c:/Users/Admin/Desktop/BCA-6sem/AI/p2.py
ABDEFC
```

3. . Write a Program to Implement 8-Puzzle problem using Python.

```
47. # -----
48. # Initial and Goal States
49. # -----
50.
51. start = [1, 2, 3,
52.         4, 0, 6,
53.         7, 5, 8]
54.
55. goal = [1, 2, 3,
56.        4, 5, 6,
57.        7, 8, 0]
58.
59. # -----
60. # Possible moves (index positions)
61. # -----
62.
63. moves = {
64.     0: [1, 3],
65.     1: [0, 2, 4],
66.     2: [1, 5],
67.     3: [0, 4, 6],
68.     4: [1, 3, 5, 7],
69.     5: [2, 4, 8],
70.     6: [3, 7],
71.     7: [4, 6, 8],
72.     8: [5, 7]
73. }
74.
75. # -----
76. # BFS Solve Function
77. # -----
78.
79. def solve(start):
80.     visited = []
81.     queue = []
82.
83.     queue.append((start, [])) # (current_state, path)
84.     visited.append(start)
85.
86.     while queue:
87.         current, path = queue.pop(0)
88.
89.         # Check if goal reached
90.         if current == goal:
91.             return path + [current]
92.
93.         zero_index = current.index(0)
94.
95.         # Try all possible moves
96.         for move in moves[zero_index]:
97.             new_state = current.copy()
98.
99.             # Swap 0 with neighbor
```

```

100.     new_state[zero_index], new_state[move] = new_state[move], new_state[zero_index]
101.
102.     if new_state not in visited:
103.         visited.append(new_state)
104.         queue.append((new_state, path + [current]))
105.
106.     return None
107.
108. # -----
109. # Print Puzzle Function
110. # -----
111.
112. def print_puzzle(state):
113.     # state is a list like [1,2,3,4,5,6,7,8,0]
114.
115.     for i in range(0, 9, 3):
116.         # range(0, 9, 3) gives: 0, 3, 6
117.
118.         # state[i:i+3] means:
119.         # start from index i
120.         # go up to (but not including) i+3
121.
122.         # Example:
123.         # i = 0 → state[0:3] → [1, 2, 3]
124.         # i = 3 → state[3:6] → [4, 5, 6]
125.         # i = 6 → state[6:9] → [7, 8, 0]
126.
127.         print(state[i:i+3]) # prints one row (3 elements)
128.
129.     print() # empty line for spacing
130.
131. # -----
132. # Main Execution
133. # -----
134.
135. solution = solve(start)
136.
137. if solution:
138.     print("Solution Found!\n")
139.
140.     for step in solution:
141.         print_puzzle(step)
142.
143.     print("Total Moves:", len(solution) - 1)
144.
145. else:
146.     print("No Solution")

```

Output :

```
PS C:\Users\Admin\Desktop\BCA-6sem\AI> & "C:\Program Files\Python313\python.exe" c:/Users/Admin/Desktop/BCA-6sem/AI/p3.py  
Solution Found!
```

```
[1, 2, 3]  
[4, 0, 6]  
[7, 5, 8]
```

```
[1, 2, 3]  
[4, 5, 6]  
[7, 0, 8]
```

```
[1, 2, 3]  
[4, 5, 6]  
[7, 8, 0]
```

```
Total Moves: 2
```

```
PS C:\Users\Admin\Desktop\BCA-6sem\AI>
```

4. Write a Program to Implement Alpha-Beta Pruning using Python.

```
147.
148. # Alpha-Beta Pruning in Python
149.
150. MAX, MIN = 1000, -1000
151.
152. def minimax(depth, nodeIndex, maximizingPlayer,
153.             values, alpha, beta):
154.
155.     # Leaf node reached
156.     if depth == 3:
157.         return values[nodeIndex]
158.
159.     # Maximizer
160.     if maximizingPlayer:
161.
162.         best = MIN
163.
164.         for i in range(2):
165.
166.             val = minimax(depth + 1,
167.                             nodeIndex * 2 + i,
168.                             False, values,
169.                             alpha, beta)
170.
171.             best = max(best, val)
172.             alpha = max(alpha, best)
173.
174.             # Pruning
175.             if beta <= alpha:
176.                 break
177.
178.         return best
179.
180.     # Minimizer
181.     else:
182.
183.         best = MAX
184.
185.         for i in range(2):
186.
187.             val = minimax(depth + 1,
188.                             nodeIndex * 2 + i,
189.                             True, values,
190.                             alpha, beta)
191.
```

```

192.         best = min(best, val)
193.         beta = min(beta, best)
194.
195.         # Pruning
196.         if beta <= alpha:
197.             break
198.
199.     return best
200.
201. # Terminal values
202. values = [3, 5, 6, 9, 1, 2, 0, -1]
203.
204. # Function Call
205. result = minimax(0, 0, True,
206.                  values, MIN, MAX)
207.
208. print("Optimal Value is :", result)
209.

```

Output :

```

PS C:\Users\Admin\Desktop\BCA-6sem\AI> & "C:\Program Files\Python313\python.exe" c:/Users/Admin/Desktop/BCA-6sem/AI/p5.py
Optimal Value is : 5

```

5. Write a program to implement forward chaining algorithm.

```
210. # Forward Chaining using ABCD
211.
212. database = ["A","B","C","D"]
213. print("Forwar Chaining ")
214. print("\n 1.A \n 2.B \n 3.C \n 4.D")
215. n=int(input("Enter the Raw Data Number: "))
216. if n == 1:
217.     print("A->B")
218. elif n == 2:
219.     print("B->C")
220. elif n == 3:
221.     print("C->D")
222. elif n == 4:
223.     print("D Goal Reached")
224. else:
225.     print("Invalid INput")
226.
227. if n>=1 and n<=4:
228.     print("Selected : ",database[n-1])
229.
```

Output :

```
PS C:\Users\Admin\Desktop\BCA-6sem\AI\pracitceing> &
AI/pracitceing/p5.py
Forwar Chaining

1.A
2.B
3.C
4.D
Enter the Raw Data Number: 1
A->B
Selected : A
PS C:\Users\Admin\Desktop\BCA-6sem\AI\pracitceing>
```


6. Write a program to implement backward chaining algorithm.

```
230. # Backward Chaining using ABCD
231.
232. database = ["A","B","C","D"]
233.
234. print("\n A \n B \n C \n D")
235.
236. goal = input("Enter the Goal :")
237.
238. if goal == "A":
239.     print("A is the Initial Fact")
240.
241. elif goal == "B":
242.     print("B <- A")
243.
244. elif goal == "C":
245.     print("C <- B")
246.     print("B <- A")
247.
248. elif goal == "D":
249.     print("D <- C")
250.     print("C <- B")
251.     print("B <- A")
252.
253. else :
254.     print("Invalid Input")
255.
256.
```

Output :

```
PS C:\Users\Admin\Desktop\BCA-6sem\AI\practiceng> &
AI/practiceng/p6.py

A
B
C
D
Enter the Goal :D
D <- C
C <- B
B <- A
```

7. Write a program to implement KNN algorithm to classify Iris dataset. Print both correct and wrong predictions.

DataSet Link : <https://github.com/pashaarshad/BCA-6sem/blob/main/AI/Iris.csv>

```
257.
258.
259.
260. # KNN Algorithm using Iris CSV Dataset
261.
262. import pandas as pd
263.
264. from sklearn.model_selection import train_test_split
265. from sklearn.neighbors import KNeighborsClassifier
266. from sklearn.metrics import accuracy_score
267.
268. # Load Dataset
269. data = pd.read_csv("Iris.csv")
270.
271. # Input and Output
272. X = data.iloc[:, 1:-1].values
273. y = data.iloc[:, -1].values
274.
275. # Split Dataset
276. X_train, X_test, y_train, y_test = train_test_split(
277.     X, y, test_size=0.2, random_state=1)
278.
279. # Create Model
280. model = KNeighborsClassifier(n_neighbors=3)
281.
282. # Train Model
283. model.fit(X_train, y_train)
284.
285. # Prediction
286. pred = model.predict(X_test)
287.
288. # Accuracy
289. print("Accuracy :", accuracy_score(y_test, pred))
290.
291. print("\nPredictions:\n")
292.
293. # Print Correct and Wrong Predictions
294. for i in range(len(y_test)):
295.
296.     print("Actual :", y_test[i],
297.         " Predicted :", pred[i], end=' ')
```

```
298.  
299.     if y_test[i] == pred[i]:  
300.         print("-> Correct")  
301.  
302.     else:  
303.         print("-> Wrong")
```

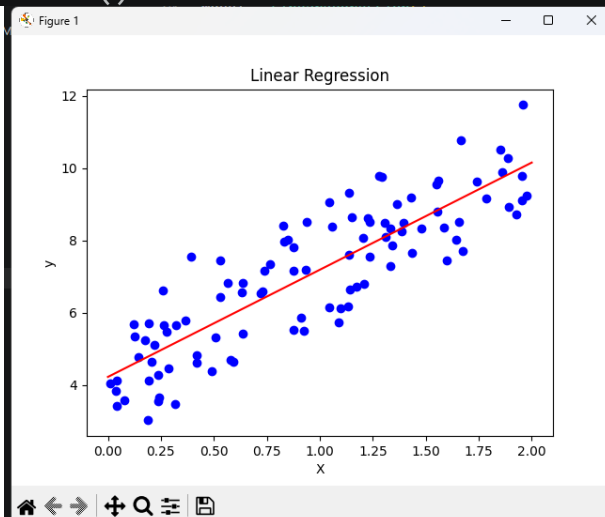
Output :

```
● PS C:\Users\Admin\Desktop\BCA-6sem\AI> & "C:\Program Files\Python313\python.exe" c:/Users  
Accuracy : 0.9666666666666667  
  
Predictions:  
  
Actual : setosa Predicted : setosa -> Correct  
Actual : versicolor Predicted : versicolor -> Correct  
Actual : versicolor Predicted : versicolor -> Correct  
Actual : setosa Predicted : setosa -> Correct  
Actual : virginica Predicted : virginica -> Correct  
Actual : versicolor Predicted : versicolor -> Correct  
Actual : virginica Predicted : virginica -> Correct  
Actual : setosa Predicted : setosa -> Correct  
Actual : setosa Predicted : setosa -> Correct  
Actual : virginica Predicted : virginica -> Correct  
Actual : versicolor Predicted : versicolor -> Correct  
Actual : setosa Predicted : setosa -> Correct  
Actual : virginica Predicted : virginica -> Correct  
Actual : versicolor Predicted : versicolor -> Correct  
Actual : versicolor Predicted : versicolor -> Correct  
Actual : setosa Predicted : setosa -> Correct  
Actual : versicolor Predicted : versicolor -> Correct  
Actual : versicolor Predicted : versicolor -> Correct  
Actual : setosa Predicted : setosa -> Correct  
Actual : setosa Predicted : setosa -> Correct  
Actual : versicolor Predicted : versicolor -> Correct  
Actual : versicolor Predicted : versicolor -> Correct  
Actual : versicolor Predicted : virginica -> Wrong  
Actual : setosa Predicted : setosa -> Correct  
Actual : virginica Predicted : virginica -> Correct
```

8. Train a random data sample using linear regression model and plot the graph.

DataSet Link : <https://github.com/pashaarshad/BCA-6sem/blob/main/AI/Iris.csv>

```
304.  
305. import numpy as np  
306. import matplotlib.pyplot as plt  
307.  
308. from sklearn.linear_model import LinearRegression  
309.  
310. # Generate some sample data  
311. np.random.seed(0)  
312.  
313. X = 2 * np.random.rand(100, 1)  
314. y = 4 + 3 * X + np.random.randn(100, 1)  
315.  
316. # Train the linear regression model  
317. model = LinearRegression()  
318. model.fit(X, y)  
  
319. # Make predictions  
320. X_new = np.array([[0], [2]])  
321. y_pred = model.predict(X_new)  
322.  
323. # Plot the data points and the linear regression line  
324.  
325. plt.scatter(X, y, color='blue')  
326. plt.plot(X_new, y_pred, color='red')  
327. plt.xlabel('X')  
328. plt.ylabel('y')  
329. plt.title('Linear Regression')  
330.  
331. plt.show()
```



Output :

9. Implement the naïve Bayesian classifier for a sample training data set stored as a .csv file. Compute the accuracy of the classifier, considering few test data sets.

DataSet Link : <https://github.com/pashaarshad/BCA-6sem/blob/main/AI/Iris.csv>

```
332. # Naive Bayes using Iris CSV Dataset
333.
334. import pandas as pd
335.
336. from sklearn.model_selection import train_test_split
337. from sklearn.naive_bayes import GaussianNB
338. from sklearn.metrics import accuracy_score, confusion_matrix
339.
340. # Load Dataset
341. data = pd.read_csv("Iris.csv")
342.
343. # Input and Output
344. X = data.iloc[:, 1:-1].values
345. y = data.iloc[:, -1].values
346.
347. # Split Dataset
348. X_train, X_test, y_train, y_test = train_test_split(
349.     X, y, test_size=0.2, random_state=1)
350.
351. # Create Model
352. model = GaussianNB()
353.
354. # Train Model
355. model.fit(X_train, y_train)
356.
357. # Prediction
358. pred = model.predict(X_test)
359.
360. # Accuracy
361. print("Accuracy :", accuracy_score(y_test, pred))
362.
363. # Confusion Matrix
364. print("\nConfusion Matrix:\n")
365.
366. print(confusion_matrix(y_test, pred))
367.
```

```
PS C:\Users\Admin\Desktop\BCA-6sem\AI> & "python Iris_classifier.py"
Accuracy : 0.9666666666666667

Confusion Matrix:

[[11  0  0]
 [ 0 12  1]
 [ 0  0  6]]
```

Output :

10. Demonstrate the working of SVM classifier for a suitable data set(e.g., iris dataset)

```
368. # SVM Classifier using Iris CSV Dataset
369.
370. import pandas as pd
371.
372. from sklearn.model_selection import train_test_split
373. from sklearn.preprocessing import StandardScaler
374.
375. from sklearn.svm import SVC
376. from sklearn.metrics import accuracy_score, confusion_matrix
377.
378. # Load Dataset
379. data = pd.read_csv("Iris.csv")
380.
381. # Input and Output
382. X = data.iloc[:, 1:-1].values
383. y = data.iloc[:, -1].values
384.
385. # Split Dataset
386. X_train, X_test, y_train, y_test = train_test_split(
387.     X, y, test_size=0.2, random_state=42)
388.
389. # Feature Scaling
390. # Below 3 lines are optional
391.
392. scaler = StandardScaler()
393.
394. X_train = scaler.fit_transform(X_train)
395. X_test = scaler.transform(X_test)
396.
397. # Create SVM Model
398. model = SVC(kernel='linear')
399.
400. # Train Model
401. model.fit(X_train, y_train)
402.
403. # Prediction
404. pred = model.predict(X_test)
405.
406. # Accuracy
407. print("Accuracy :", accuracy_score(y_test, pred))
408. # Confusion Matrix
409. print("\nConfusion Matrix:\n")
410.
411. print(confusion_matrix(y_test, pred))
412.
```

Output :

```
PS C:\Users\Admin\Desktop\BCA-6sem\AI> & "C:\Program Files\Python311\python.exe" C:\Users\Admin\Desktop\BCA-6sem\AI\main.py
Accuracy : 0.9666666666666667

Confusion Matrix:

[[10  0  0]
 [ 0  8  1]
 [ 0  0 11]]

PS C:\Users\Admin\Desktop\BCA-6sem\AI>
```